Documentation — Phase 2: WHY

Aviation — U.S. Flight Delay Network

Justifying Technical Decisions from Phase 1

Purpose: Justify the technical decisions made in Phase 1. Every choice in the stack is traceable to a specific reason grounded in the dataset and the research question.

Team

Role	Name
Team Lead	Akhmedova Muslima
Member 2	O'ralov Ilyos
Member 3	Aliqulov Davron
Member 4	Gulamov Abdulaziz
Member 5	Boymuhamedov Komronbek

---

1. Why This Scenario?

The Aviation — U.S. Flight Delay Network scenario was selected for three interconnected reasons: data richness, analytical novelty, and real-world impact.

First, the dataset is among the most accessible and well-documented public transportation datasets available, with 5.8 million rows and 40 columns covering the full year of 2015 — large enough to surface statistically meaningful patterns but manageable within a single-semester project. Second, the core question — which airports act as 'super-spreaders' of delays — maps naturally onto graph theory, allowing us to build skills in network analysis that go beyond standard descriptive statistics or regression. Third, the practical stakes are high: identifying the five airports that propagate the most delay minutes could directly inform FAA buffer-time policy, airline scheduling, and passenger communication systems. Unlike other scenarios, this one lets us combine data engineering (cleaning 5.8M rows efficiently), network science (directed weighted graphs), and stakeholder communication (visual recommendation reports) in a single pipeline.

---

2. Methodological Justification

2.1 Why This Analytical Approach?

We chose Directed Weighted Graph Analysis (NetworkX DiGraph) over alternatives such as regression modelling or simple ranking because the business question is inherently relational: we are not asking 'which airport has the most delays in isolation?' but rather 'which airport causes the most delays downstream throughout the rest of the network?' A regression model on individual airports would miss the propagation structure entirely. A simple average-delay ranking would treat each airport independently and could mislead — an airport with moderate average delays but hundreds of high-traffic outbound routes may cause far more total disruption than a single chronically late airport with few connections.

Weighted Out-Strength Centrality is the correct centrality metric here because our edges are directional (delays travel Origin → Destination) and our edge weights represent real delay volume (sum of arrival delay minutes on delayed routes). In-strength centrality would measure which airports receive delays, not which ones originate them. Betweenness centrality would identify network bottlenecks but not delay generators. Out-strength precisely captures 'how many delay minutes does this airport push outward into the system,' which is exactly the spillover definition we need.

2.2 Why This Aggregation Level?

- **Route level (Origin → Destination pair)**, not flight level: aggregating by route collapses 5.8M rows into a manageable graph edge list while preserving the directional structure. Flight-level edges would create a multigraph with millions of parallel edges, making centrality computation unnecessary slow and memory-intensive.
- **Annual aggregate (full year 2015)**, not monthly: our goal is structural network identification, not seasonal trend analysis. Seasonal splits would fragment the graph and make some routes too sparse to rank reliably. A full-year aggregate produces stable edge weights.
- **Airport level (IATA code)**, not airline level: the same physical airport hosts multiple airlines. A congestion or weather event at ORD affects all carriers departing from it; attributing delays to the airport rather than to a carrier correctly models the physical reality of airspace congestion.

2.3 Why These Features and Not Others?

Columns kept and reason for each:

Column	Reason for Keeping
ORIGIN_AIRPORT	Source node in the directed graph
DESTINATION_AIRPORT	Target node in the directed graph
DEPARTURE_DELAY	Filter threshold — identifies flights that departed significantly late (>15 min)
ARRIVAL_DELAY	Edge weight — quantifies how many minutes of delay reached the destination

FLIGHT_NUMBER	De-duplication key — used to remove exact duplicate records
MONTH / DAY_OF_WEEK	Reserved for the time-series visualization (Stage 4)

Columns dropped and reason for each:

Column	Reason for Dropping
AIRLINE	Delay is attributed to the airport, not the carrier, in this model
SCHEDULED_DEPARTURE / SCHEDULED_ARRIVAL	Raw times not needed once DEPARTURE_DELAY and ARRIVAL_DELAY are present
TAIL_NUMBER / FLIGHT_NUMBER (after dedup)	Aircraft identifiers irrelevant to route-level aggregation
WHEELS_OFF / WHEELS_ON / TAXI_IN / TAXI_OUT	Sub-components of delay; ARRIVAL_DELAY already captures the total
CANCELLATION_REASON / CANCELLED / DIVERTED	Cancelled/diverted flights removed in cleaning — these columns become all-null
AIR_SYSTEM_DELAY / SECURITY_DELAY etc.	Delay cause breakdown is useful for future work but out of scope for network centrality

2.4 Limits of the Method

Two cases where our conclusions would be invalid:

1. The model assumes that delays on a route are caused by the origin airport. In reality, a delayed aircraft arriving late at an intermediate stop (rotation delay) can cause the next departure to be late even if the origin airport is operating normally. Our directed graph will over-attribute blame to high-traffic origin hubs and under-attribute it to the upstream airports that originally triggered the chain. A more accurate model would track individual tail numbers across legs — a complexity beyond our current scope.
2. The 2015 dataset reflects a specific year's traffic volume, weather patterns, and airline scheduling decisions. Structural findings (e.g. that ATL or ORD rank highest) may not generalise to post-pandemic traffic in 2024–2025, where route networks have been substantially restructured and some previously dominant hubs have reduced capacity. Our recommendations should be labelled explicitly as 2015-data conclusions.

---

3. Tooling Selection

3.1 Why This Library / Package?

- **Pandas over raw Python loops:** the dataset has 5.8M rows. A Python for-loop iterating row-by-row would run in the order of minutes for simple operations. Pandas' vectorised C-backed operations complete the same tasks in seconds. dtype optimization (e.g. specifying int16 for delay columns) further reduces RAM consumption from ~3.5 GB (default float64) to under 1 GB, avoiding out-of-memory errors in Colab's 12 GB environment.
- **NetworkX over igraph or graph-tool:** NetworkX is pure Python, ships with Colab, and has a well-documented DiGraph class with built-in weighted degree methods. igraph is faster for very large graphs (millions of nodes), but our graph has at most ~350 airport nodes and ~4,500 route edges — a scale where NetworkX runs sub-second and the simpler API reduces debugging time. graph-tool requires a system-level install not available on standard Colab.
- **Kaggle API over manual download:** the flights.csv file is 592 MB. A manual download link embedded in a notebook would break if the dataset URL changes and prevents automated re-runs. The Kaggle API call is one reproducible line that always fetches the authoritative version.

3.2 Why This Visualization Tool?

- **Matplotlib (base) + Seaborn (statistical charts):** Seaborn is chosen for the bar chart (Top 5 airports) and heatmap (origin-destination delay matrix) because it produces publication-quality statistical graphics with minimal code, handles axis labelling and colour palettes automatically, and integrates directly with Pandas DataFrames. Matplotlib is used as the rendering backend for the NetworkX graph layout, because NetworkX's draw() API targets Matplotlib directly.
- **Plotly was considered but rejected** for the primary deliverables: interactive HTML charts are harder to embed in a static report PDF and add a dependency that Colab must install at runtime. For an audience of aviation planners reading a written deliverable, static PNG exports are sufficient and more portable.

3.3 Why This Storage / Format?

- **CSV over Parquet or SQLite:** the cleaned dataset (~2M rows after filtering) fits comfortably in a single CSV file under 300 MB. Parquet would offer faster columnar reads but adds a pyarrow dependency and is less transparent for peer review — a mentor cannot open a Parquet file directly in Excel to spot-check values. SQLite would add query overhead without benefit, as all analysis is done in-memory with Pandas and NetworkX.
- **Jupyter Notebook (Google Colab) over standalone .py script:** the deliverable requires inline visualizations, narrative commentary, and step-by-step reproducibility for mentor review. A .py script would require separate figure files and a README. Colab combines code, output, and prose in one shareable link with zero local setup for the reviewer.
- **Google Drive / Colab link over GitHub for the final submission:** the 592 MB raw dataset cannot be committed to a standard GitHub repository (100 MB file limit). Colab notebooks stored in Drive link directly to the Kaggle API download, avoiding large binary file storage entirely.

3.4 Why This Validation Reference?

- **NetworkX built-in weighted degree functions** are used as the primary validation reference for centrality scores. We will cross-check our manual aggregation (groupby + sum) against NetworkX's G.out_degree(weight='weight') to confirm both methods produce identical rankings.
- **BTS Air Travel Consumer Reports (2015)** published by the U.S. Department of Transportation are used as a sanity check for our Top 5 list. If our model identifies airports that do not appear in publicly reported 2015 delay hotspots (e.g. ORD, ATL, DFW, LAX), that discrepancy would signal a data processing error rather than a genuine finding.

---

4. What Did We NOT Pick — and Why?

3. Machine Learning (e.g. Random Forest or XGBoost for delay prediction) — We considered a predictive model but rejected it because the business question is descriptive and structural ('which airports spread delays?'), not predictive ('will this specific flight be delayed?'). A classifier would require train/test splits, feature engineering, and hyperparameter tuning that add complexity without answering the core network-propagation question.
4. Gephi for network visualization — Gephi is a dedicated graph visualization tool that produces more polished network diagrams than NetworkX/Matplotlib. We rejected it because it requires a desktop install, cannot be run inside a Colab notebook, and would break the single-notebook reproducibility requirement. NetworkX with a spring-layout rendering is sufficient to show the Top 5 nodes and their connections within the submission format.

---

5. Submission Checklist

- [x] Phase 1 doc completed and signed off
- [x] This Phase 2 doc completed
- [ ] Both pushed to group repo at module-2/class_6/lab-scenarios/10_flight_delay_network/
- [ ] Mentor approval received before any code is written

Mentor signature / date: _____